EXP NO: 14 code using RAW sockets to implement packet sniffing

4.9.25

Code:

AIM

To write a program using RAW socket capture and display network packets parsing throwing the network interface

ALGORITHM:

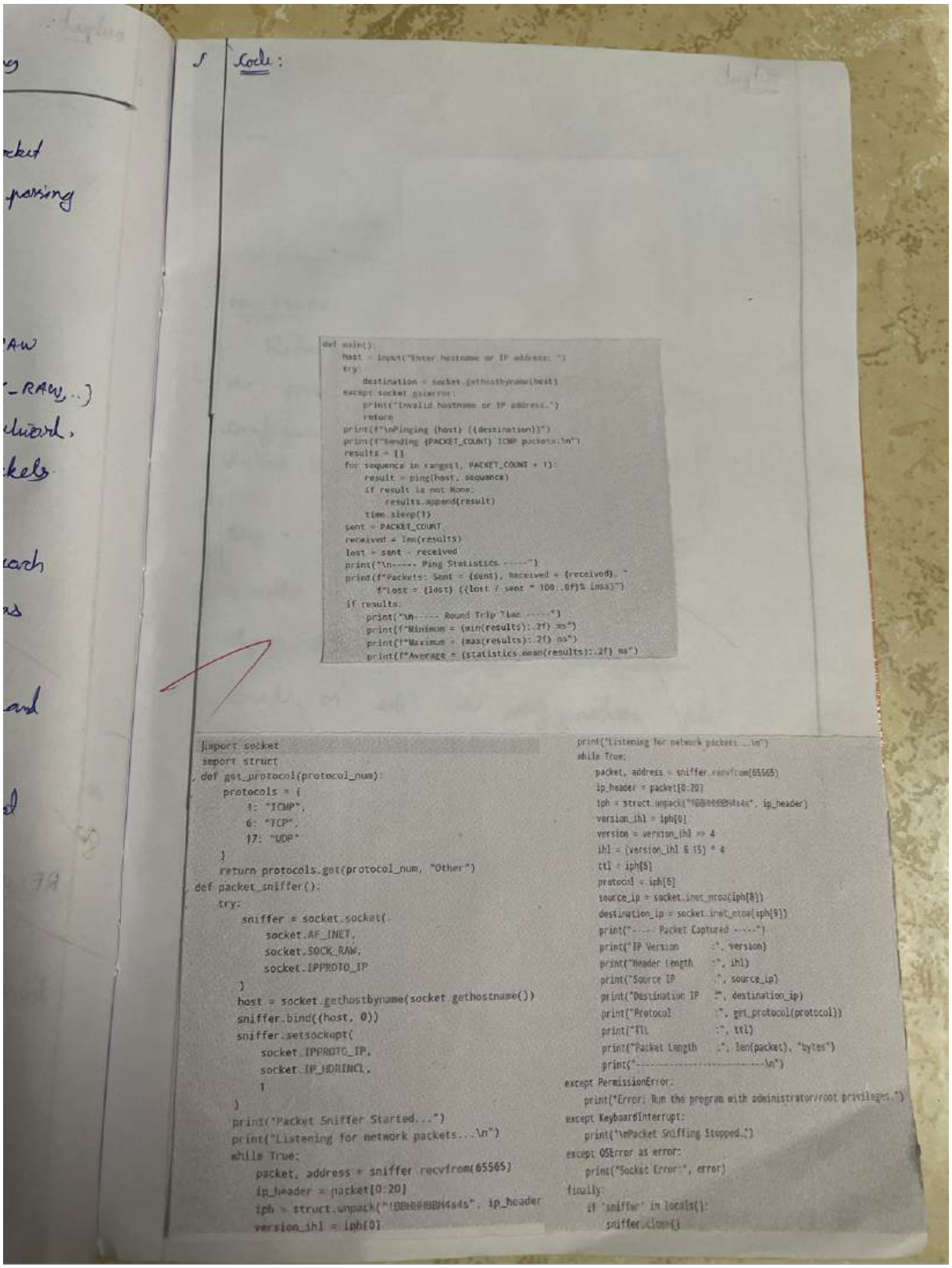
i) Run the program, and create a RAW packet using socket (AF_PACKET, SOCKET_RAW, ..)

ii) Bind the socket to the required network interface then continuously receive packets from the network

iii) Extract the Ethernet header from each packet and identify protocol type, such as IPV4, ARP, or IPV6

iv) For IPV4 packets, extract the source and the destination IP addresses.

v) Display the packet information. Repeat until the user terminates the program.



Code:

```
def main():
    host = input("Enter hostname or IP address: ")
    try:
        destination = socket.gethostbyname(host)
    except socket.gaierror:
        print("Invalid hostname or IP address.")
        return
    print(f"\nPinging {host} [{destination}]")
    print(f"Sending {PACKET_COUNT} ICMP packets:\n")
    results = []
    for sequence in range(1, PACKET_COUNT + 1):
        result = ping(host, sequence)
        if result is not None:
            results.append(result)
        time.sleep(1)
    sent = PACKET_COUNT
    received = len(results)
    lost = sent - received
    print("\n----- Ping Statistics -----")
    print(f"Packets: Sent = {sent}, Received = {received}, "
          f"Lost = {lost} ({lost / sent * 100:.0f}% loss)")
    if results:
        print("\n----- Round Trip Time -----")
        print(f"Minimum = {min(results):.2f} ms")
        print(f"Maximum = {max(results):.2f} ms")
        print(f"Average = {statistics.mean(results):.2f} ms")
```

```
import socket
import struct
def get_protocol(protocol_num):
    protocols = {
        1: "ICMP",
        6: "TCP",
        17: "UDP"
    }
    return protocols.get(protocol_num, "Other")
def packet_sniffer():
    try:
        sniffer = socket.socket(
            socket.AF_INET,
            socket.SOCK_RAW,
            socket.IPPROTO_IP
        )
        host = socket.gethostbyname(socket.gethostname())
        sniffer.bind((host, 0))
        sniffer.setsockopt(
            socket.IPPROTO_IP,
            socket.IP_HDRINCL,
            1
        )
        print("Packet Sniffer Started...")
        print("Listening for network packets...\n")
        while True:
            packet, address = sniffer.recvfrom(65565)
            ip_header = packet[0:20]
            iph = struct.unpack("!BBHHBBH4s4s", ip_header
            version_ihl = iph[0]
            print("Listening for network packets...\n")
            while True:
                packet, address = sniffer.recvfrom(65565)
                ip_header = packet[0:20]
                iph = struct.unpack("!BBHHBBH4s4s", ip_header)
                version_ihl = iph[0]
                version = version_ihl >> 4
                ihl = (version_ihl & 15) * 4
                ttl = iph[5]
                protocol = iph[6]
                source_ip = socket.inet_ntoa(iph[8])
                destination_ip = socket.inet_ntoa(iph[9])
                print("----- Packet Captured -----")
                print("IP Version      :", version)
                print("Header Length   :", ihl)
                print("Source IP       :", source_ip)
                print("Destination IP  :", destination_ip)
                print("Protocol        :", get_protocol(protocol))
                print("TTL             :", ttl)
                print("Packet Length   :", len(packet), "bytes")
                print("-----------------------------\n")
        except PermissionError:
            print("Error: Run the program with administrator/root privileges.")
        except KeyboardInterrupt:
            print("\nPacket Sniffing Stopped.")
        except OSError as error:
            print("Socket Error:", error)
        finally:
            if 'sniffer' in locals():
                sniffer.close()
```

output:

```
Packet Sniffer Started...
Listening for network packets...

----- Packet Captured -----
IP Version        : 4
Header Length     : 20
Source IP         : 192.168.1.5
Destination IP    : 142.250.183.14
Protocol          : TCP
TTL               : 64
Packet Length     : 60 bytes
---------------------------

----- Packet Captured -----
IP Version        : 4
Header Length     : 20
Source IP         : 192.168.1.5
Destination IP    : 8.8.8.8
```

RESULT:

The code to implement RAW sockets to implement packet sniffing has been completed.

EXP NO: 15

AIM

PROCEDURE